

Real Inverse Asymptotics at a Singular Endpoint

Power–logarithmic expansions, irrational exponents,
branch selection, and a finite inversion algorithm

A research article and implementation prepared for
Vladimir Reshetnikov

September 7, 2026 • Package version 1.0.0

The principal result. For $f(x) = cx^p(1 + \sum_{j=1}^r x^{\delta_j} P_j(\log x))$, with $c, p, \delta_j > 0$, the positive inverse has explicitly computable polynomial coefficients on the locally finite support $1 + \mathbb{N}\delta$. The construction works without analyticity at zero, merges coincident exponents exactly, and supplies a logarithm-aware remainder at every power cutoff.

Validation boundary. Seventy-nine independent symbolic and 250-digit numerical checks were run successfully in Python. The Wolfram package and a 33-test native suite are included. No native Wolfram execution is claimed: the connected evaluator returned an unavailable-endpoint error in this session. Formal residual checks in the package are not numerical interval certificates.

Abstract

The inverse of $x + x^2(1 + \log x)$ at 0^+ is not an ordinary Taylor series, and the inverse of $x + x^{\sqrt{2}}$ requires irrational exponents. Both are instances of an elementary but useful inversion calculus for finite power–logarithmic perturbations of a positive monomial. We prove existence and selection of the positive real branch, a closed multi-index coefficient formula, local convergence of the resulting generalized series for exact finite models, and quantitative asymptotic remainder orders for arbitrary weighted truncations. A finite border of an order ideal determines the first possible omitted power and a safe logarithmic degree, even when generator weights collide. A separate Newton algorithm in a truncated logarithmic jet algebra gives the same inverse, and forward residuals transfer to inverse errors after the correct derivative rescaling. We also treat finite forward expansions with differentiated remainder bounds, explain why a bound on the forward function alone need not preserve invertibility, and state the boundaries of the implemented class. The accompanying Wolfram Language package contains both coefficient and Newton engines, exact exponent ordering, explicit failure cases, residual inspection, and a more general auxiliary-parameter inversion utility. Complete examples, proofs, reproducible verification programs, a native regression suite, and the full package listing accompany the article.

Contents

1	The problem and the answer	1
1.1	The two examples	1
1.2	Immediate use of the package	1
2	Selecting the real branch before expanding	2
2.1	Global uniqueness in the original logarithmic example	2
2.2	The general finite model	2
3	The algebra of complete logarithmic blocks	3
3.1	Support and dominance	3
3.2	A filtered algebra rather than a misleading Taylor series	4
4	Lagrange inversion at a moving positive point	4
4.1	Relation to the supplied repository	5
5	A closed coefficient formula for every block	5
5.1	Low orders and nonlogarithmic specialization	6
6	Why the generalized series actually converges	7
7	Weighted truncation and a finite-border remainder	8
7.1	Strict cutoff semantics	8
7.2	Complexity and exact ordering	9
8	A second engine: Newton inversion in finite jets	9
9	Residuals, actual inverse errors, and certification	10
10	Inverting a finite forward expansion with a remainder	11

11 The logarithmic example to arbitrary order	12
12 The irrational-power example and its convergence radius	13
13 Mixed generators, collisions, and a nonunit core	14
13.1 A genuinely mixed scale	14
13.2 Coincident weights must be added	14
13.3 The core coefficient and exponent both matter	15
14 The Wolfram Language package	15
14.1 Loading and the principal function	15
14.2 Accepted input and deliberate failures	15
14.3 Options	16
14.4 Result fields and representation	16
14.5 Residual inspection and generic perturbation inversion	17
14.6 Relationship to built-in series operations	17
15 Verification record and reproducibility	18
15.1 What was executed	18
15.2 What was not executed	19
15.3 Rebuilding the article	19
16 Boundaries and extensions of the theory	19
16.1 Finite jets extracted from a more general forward function	19
16.2 Other centers and the point at infinity	20
16.3 Logarithmic leading cores and deeper transseries	20
16.4 Parameter limits and large-order computation	20
17 Conclusion	21
A Distribution map and source provenance	21
B Complete Wolfram Language package	22

1 The problem and the answer

1.1 The two examples

Vladimir Reshetnikov's question, posted on December 11, 2020 and later updated with an irrational-power example, asks for the real inverse near zero of

$$f(x) = x + x^2(1 + \log x), \quad x > 0, \quad (1.1)$$

and for an analogous expansion of the inverse of

$$f_\alpha(x) = x + x^\alpha, \quad \alpha = \sqrt{2}. \quad (1.2)$$

The uploaded Markdown copy and the public question agree on these two problems [1]. The reported computer-algebra expressions in that question are historical observations, not tests of the present version of Mathematica.

Write $L = \log y$. The first answer begins

$$\begin{aligned} g(y) = & y - y^2(1 + L) + y^3(2L^2 + 5L + 3) \\ & - y^4 \left(5L^3 + \frac{41}{2}L^2 + 27L + \frac{23}{2} \right) \\ & + y^5 \left(14L^4 + \frac{241}{3}L^3 + \frac{335}{2}L^2 + 151L + \frac{299}{6} \right) \\ & + O(y^6(1 + |L|)^5), \quad y \longrightarrow 0^+. \end{aligned} \quad (1.3)$$

In particular, the correct two-block statement is

$$g(y) = y - y^2(1 + \log y) + O(y^3(1 + |\log y|)^2). \quad (1.4)$$

It is *not* valid to replace this remainder by the ordinary analytic Landau bound $O(y^3)$. Indeed, the next coefficient is $2L^2 + 5L + 3$, which is unbounded. One may instead write $O(y^{3-\varepsilon})$ for every fixed $\varepsilon > 0$, but that loses useful information.

For the second example,

$$g_\alpha(y) = \sum_{n=0}^{\infty} b_n(\alpha) y^{1+n(\alpha-1)}, \quad (1.5)$$

$$b_n(\alpha) = \frac{(-1)^n}{n!} (\alpha n)^{n-1} \quad (n \geq 1), \quad b_0(\alpha) = 1. \quad (1.6)$$

At $\alpha = \sqrt{2}$, this gives exactly

$$\begin{aligned} g_{\sqrt{2}}(y) = & y - y^{\sqrt{2}} + \sqrt{2} y^{2\sqrt{2}-1} - \frac{6 - \sqrt{2}}{2} y^{3\sqrt{2}-2} \\ & + O(y^{4\sqrt{2}-3}). \end{aligned} \quad (1.7)$$

Both formulas follow from the same theorem below. In fact, for the exact finite models considered here, sufficiently small positive y gives convergent generalized series, not merely formal ones.

1.2 Immediate use of the package

Extract the distribution and evaluate the following, with the symbols $\mathbf{x}$, $\mathbf{y}$, and $\mathbf{z}$ unassigned:

```

Get["path/to/real-inverse-series/Kernel/RealInverseSeries.wl"];
Clear[x, y, z];

r1 = RealInverseSeries[x + x^2 (1 + Log[x]), {x, y}, 6];
r1["Expansion"]
r1["Remainder"]

r2 = RealInverseSeries[x + x^Sqrt[2], {x, z}, 4 Sqrt[2] - 3];
r2["Expansion"]
r2["Remainder"]

```

The third argument is a *strict power cutoff*: retain all complete logarithmic blocks whose exponent in the output variable is smaller than that number. It is not a count of monomials and not a count of auxiliary-parameter orders. Thus cutoff 6 in the first call retains the five blocks displayed in (1.3); cutoff $4\sqrt{2} - 3$ in the second call retains precisely the four terms in (1.7). The returned association keeps the ordinary expression separate from its mathematically valid remainder scale.

2 Selecting the real branch before expanding

2.1 Global uniqueness in the original logarithmic example

Proposition 2.1. *The function in (1.1) is a strictly increasing bijection $(0, \infty) \rightarrow (0, \infty)$. Its inverse satisfies $g(y) \sim y$ at 0^+ . It extends to a C^1 function at zero with $g(0) = 0$ and $g'(0) = 1$, but has no C^2 extension there.*

Proof. Direct differentiation gives

$$f'(x) = 1 + x(3 + 2 \log x), \quad f''(x) = 5 + 2 \log x.$$

The function $x(3 + 2 \log x)$ has its global minimum at $x = e^{-5/2}$, with value $-2e^{-5/2}$. Consequently

$$f'(x) \geq 1 - 2e^{-5/2} > 0 \quad (x > 0).$$

Also $f(0^+) = 0$ and $f(x) \rightarrow \infty$ as $x \rightarrow \infty$. The intermediate value theorem and strict monotonicity prove the bijection and positivity. Since $f(x)/x = 1 + x(1 + \log x) \rightarrow 1$, inversion gives $g(y)/y \rightarrow 1$. For positive arguments, $g'(y) = 1/f'(g(y)) \rightarrow 1$. Finally,

$$g''(y) = -\frac{f''(g(y))}{f'(g(y))^3} \longrightarrow +\infty.$$

Thus the second derivative cannot extend continuously to zero. □

A complex inverse branch through a nonzero complex solution of $f(x) = 0$ is a different local object. An expansion with a nonzero complex constant term cannot represent the branch characterized by $g(y) > 0$ and $g(y) \rightarrow 0$. The obstacle is not a missing choice of a Taylor coefficient: the correct real branch is nonanalytic at the endpoint itself.

2.2 The general finite model

Our input class is

$$f(x) = cx^p (1 + R(x)), \quad R(x) = \sum_{j=1}^r x^{\delta_j} P_j(\log x), \quad (2.1)$$

where $c, p, \delta_j > 0$ are fixed real numbers and each P_j is a nonzero real polynomial. Repeated values of δ_j may be merged at the input stage. The case $r = 0$ is the exact monomial inverse. Unless stated otherwise, (2.1) is an exact identity on a positive interval.

Lemma 2.2 (Power beats logarithm). *For every $a > 0$ and fixed real b , $x^a(1 + |\log x|)^b \rightarrow 0$ as $x \rightarrow 0^+$. In particular $R(x) = o(1)$ and $xR'(x) = o(1)$.*

Proof. Put $x = e^{-t}$, so $t \rightarrow \infty$. The first assertion is $e^{-at}(1+t)^b \rightarrow 0$, which follows, for example, from repeated l'Hôpital differentiation when b is bounded above by an integer. For a single term,

$$x \frac{d}{dx} (x^\delta P(\log x)) = x^\delta (\delta P + P')(\log x),$$

and the same limit applies. There are only finitely many terms. $\square$

Theorem 2.3 (The positive inverse germ). *For (2.1), there is $x_0 > 0$ such that f is positive and strictly increasing on $(0, x_0)$. Its inverse g on $(0, f(x_0))$ is unique there and satisfies*

$$s = (y/c)^{1/p}, \quad L = \log s, \quad \frac{g(y)}{s} \rightarrow 1. \quad (2.2)$$

Proof. Positivity follows from $1 + R(x) \rightarrow 1$. Differentiation gives

$$f'(x) = c p x^{p-1} \left[1 + \sum_j x^{\delta_j} \left(\left(1 + \frac{\delta_j}{p} \right) P_j(\log x) + \frac{1}{p} P'_j(\log x) \right) \right]. \quad (2.3)$$

The bracket tends to 1, so the derivative is positive eventually. The function tends to zero. The intermediate value theorem gives the inverse, and its defining equation yields $g(y)/s = (1 + R(g(y)))^{-1/p} \rightarrow 1$. $\square$

The theorem selects a *germ*: a function defined for sufficiently small positive arguments, with two functions identified if they agree on a smaller such interval. It does not assert global monotonicity of every finite model. Negative correction coefficients are allowed; only the leading coefficient and exponent must be positive.

3 The algebra of complete logarithmic blocks

3.1 Support and dominance

Set

$$S = \{\mathbf{n} \cdot \boldsymbol{\delta} : \mathbf{n} \in \mathbb{N}^r\}, \quad \mathbf{n} \cdot \boldsymbol{\delta} = \sum_j n_j \delta_j. \quad (3.1)$$

Here $\mathbb{N}$ includes zero. A *block* is $s^d Q(L)$ with $Q \in \mathbb{R}[L]$. At zero, blocks of smaller power dominate blocks of larger power, regardless of their fixed polynomial degrees. Within one block, higher powers of $|L|$ dominate lower ones. It is usually clearer and less error-prone to retain the whole polynomial at each exponent.

Lemma 3.1 (Local finiteness). *For every $A > 0$, the set*

$$\mathcal{I}_A = \{\mathbf{n} \in \mathbb{N}^r : \mathbf{n} \cdot \boldsymbol{\delta} < A\} \quad (3.2)$$

is finite. Every element of S has only finitely many multi-index representations, and $S \cap [0, A)$ is finite.

Proof. Each $n_j < A/\delta_j$, so $\mathcal{I}_A$ lies in a finite rectangular box. For representations of a fixed weight d , use a box with $A > d$. $\square$

It is important not to confuse this nonnegative semigroup with the group $\sum_j \mathbb{Z}\delta_j$. An irrationally generated group may be dense, but the positive semigroup here remains locally finite. Irrational exponents therefore create no infinite computation below a finite cutoff.

If two different indices have the same weight, their polynomial coefficients must be added. For example, gaps $1/2$ and 1 satisfy $2(1/2) = 1$; the contributions indexed by $(2, 0)$ and $(0, 1)$ belong to the same output block.

3.2 A filtered algebra rather than a misleading Taylor series

Consider formal sums

$$U = \sum_{d \in S} s^d Q_d(L), \quad Q_d \in \mathbb{R}[L]. \quad (3.3)$$

Addition is coefficientwise; multiplication uses the finite Cauchy sums at every bounded weight. Terms of weight at least A form an ideal, and truncation modulo that ideal gives a finite logarithmic jet. A jet is stored as a list of pairs (d, Q_d) , ordered by exact comparison of d . No floating-point approximation of a radical exponent enters the ordering. The valuation of a nonzero jet is its smallest weight with a nonzero polynomial coefficient; the zero jet has valuation $+\infty$.

If U has strictly positive support, then

$$(1 + U)^a = \sum_{k \geq 0} \binom{a}{k} U^k, \quad \log(1 + U) = \sum_{k \geq 1} \frac{(-1)^{k+1}}{k} U^k$$

are well-defined formal sums: below a fixed cutoff only finitely many powers can contribute. A scalar $a \neq 0$ plus a positive-order jet has a reciprocal by the geometric series. We do not claim that all nonzero polynomial-logarithmic leading blocks are units in this algebra.

The logarithmic coordinate is initially formal. Its connection to actual functions is justified by the analytic theorem in [section 6](#), not by pretending that L is bounded as $s \rightarrow 0$.

Lemma 3.2 (Differential operator on a block). *For real a and polynomial Q ,*

$$\frac{d^k}{ds^k} (s^a Q(\log s)) = s^{a-k} \prod_{j=0}^{k-1} (a - j + \partial_L) Q(L). \quad (3.4)$$

All factors commute.

Proof. The case $k = 1$ is the product and chain rules. Applying the same identity inductively reduces the power by one and appends the next operator. Each factor is a polynomial in ∂_L , so the factors commute. $\square$

The derivative term ∂_L is essential. Treating $\log x$ as a frozen coefficient when composing or inverting would omit the $5L + 3$ correction in the third block of [\(1.3\)](#).

4 Lagrange inversion at a moving positive point

Classical Lagrange inversion has many equivalent forms; see [\[2, 3, 4\]](#). For this problem the useful version is an expansion in an *auxiliary perturbation parameter*, at each fixed positive y , not a Taylor expansion of the inverse at the singular point $y = 0$. This distinction is the central analytic step.

Theorem 4.1 (Auxiliary-parameter inverse). *Let F_0 be analytic near a point x_* with $F'_0(x_*) \neq 0$ and let $\phi = F_0^{-1}$ be the corresponding local inverse near $y_* = F_0(x_*)$. Let h be analytic near x_* . The branch of*

$$F_0(x) + \varepsilon h(x) = y$$

that equals $\phi(y)$ at $\varepsilon = 0$ is analytic in ε near zero and satisfies

$$x = \phi(y) + \sum_{N \geq 1} \frac{(-\varepsilon)^N}{N!} \left(\frac{d}{dy} \right)^{N-1} [\phi'(y) h(\phi(y))^N]. \quad (4.1)$$

Proof. Set $v = F_0(x)$ and $H(v) = h(\phi(v))$. The equation becomes $v + \varepsilon H(v) = y$, and the required answer is $\phi(v)$. Choose a small positively oriented circle Γ around y on which all functions are analytic and $|\varepsilon H(z)| < |z - y|$. Rouché's theorem gives one zero v inside the circle. By the residue theorem,

$$\phi(v) = \frac{1}{2\pi i} \int_{\Gamma} \phi(z) \frac{1 + \varepsilon H'(z)}{z - y + \varepsilon H(z)} dz.$$

Expand the denominator geometrically, uniformly on the circle. For $N \geq 1$, the coefficient of ε^N is

$$\frac{(-1)^N}{2\pi i} \int_{\Gamma} \left[\frac{\phi H^N}{(z - y)^{N+1}} - \frac{\phi H^{N-1} H'}{(z - y)^N} \right] dz.$$

The integral of the derivative of $\phi H^N / (z - y)^N$ vanishes, so this is

$$\frac{(-1)^N}{N} \frac{1}{2\pi i} \int_{\Gamma} \frac{\phi'(z) H(z)^N}{(z - y)^N} dz = \frac{(-1)^N}{N!} \partial_y^{N-1} (\phi' H^N)(y).$$

The constant coefficient is $\phi(y)$. This proves the formula and its local analytic convergence. Analytic dependence also follows from the implicit function theorem. $\square$

For $F_0(x) = x$, the formula reduces to

$$g(y) = y + \sum_{N \geq 1} \frac{(-1)^N}{N!} \frac{d^{N-1}}{dy^{N-1}} h(y)^N, \quad (4.2)$$

provided evaluation at $\varepsilon = 1$ is justified. Neither [theorem 4.1](#) alone nor the existence of all derivatives at positive y is enough to justify that substitution uniformly as $y \rightarrow 0$. The next two sections supply the missing control for the finite class (2.1).

4.1 Relation to the supplied repository

The companion volume *Combinatorial Transseries and Their Inverses* in the supplied repository states the phase-coordinate version of (4.1), a scaled Lagrange construction, and residual-based inverse error transport [5]. Its discussion of action collisions is the exponential counterpart of the power-weight collisions here. The canonical volume's README explicitly distinguishes operator and coefficient forms of Lagrange inversion and scopes its formalization claims [6].

The present article specializes that architecture to a zero-endpoint power-logarithmic input class. Its practical additions are the polynomial operator formula below, the finite-border logarithmic remainder theorem, and a concrete input/parser/truncation contract with two implemented engines. The underlying Lagrange principle is classical; no priority claim for the general inversion identity is intended. The repository's relevant companion sections and README were consulted, not its entire large formalization corpus. No Lean verification of this package is asserted.

5 A closed coefficient formula for every block

For a multi-index $\mathbf{n} = (n_1, \dots, n_r)$ define

$$N = |\mathbf{n}| = \sum_j n_j, \quad d = \mathbf{n} \cdot \boldsymbol{\delta}, \quad \mathbf{n}! = \prod_j n_j!, \quad Q_{\mathbf{n}}(L) = \prod_j P_j(L)^{n_j}.$$

An empty product is 1.

Theorem 5.1 (All-order power-log inverse). For (2.1), define $C_0(L) = 1$. For $N \geq 1$ set

$$C_n(L) = \frac{(-1)^N}{p^N n!} \prod_{k=1}^{N-1} (1 + d + pk + \partial_L) Q_n(L). \quad (5.1)$$

Then the positive inverse has the generalized expansion

$$g(y) = s \sum_{\mathbf{n} \in \mathbb{N}^r} s^{\mathbf{n} \cdot \delta} C_{\mathbf{n}}(\log s), \quad s = (y/c)^{1/p}. \quad (5.2)$$

For sufficiently small positive s the sum converges absolutely. At a fixed weight d , all $C_{\mathbf{n}}$ with $\mathbf{n} \cdot \delta = d$ are added. The degree bound is

$$\deg C_{\mathbf{n}} \leq \sum_j n_j \deg P_j. \quad (5.3)$$

Proof of the coefficient identity. Temporarily replace the perturbation by $\sum_j \varepsilon_j h_j(x)$ with $h_j(x) = cx^{p+\delta_j} P_j(\log x)$. Apply theorem 4.1 with $F_0(x) = cx^p$ and expand the N th power by the multinomial theorem. The coefficient of $\varepsilon^{\mathbf{n}}$ is

$$\frac{(-1)^N}{n!} \left(\frac{d}{dy} \right)^{N-1} \left[\phi'(y) \prod_j h_j(\phi(y))^{n_j} \right], \quad \phi(y) = s. \quad (5.4)$$

Now

$$\frac{d}{dy} = \frac{1}{cp} s^{1-p} \frac{d}{ds}, \quad \phi'(y) \prod_j h_j(\phi(y))^{n_j} = \frac{c^{N-1}}{p} s^{p(N-1)+d+1} Q_{\mathbf{n}}(L).$$

Each differentiation lowers the s -power by p and acts on the polynomial by that current power plus ∂_L . After $N-1$ differentiations, the scalar factor is p^{-N} , the remaining power is s^{d+1} , and the operator factors are $1 + d + pk + \partial_L$ for $k = 1, \dots, N-1$. This proves (5.1). Differentiation of a polynomial cannot increase its degree, giving (5.3). The convergence and equality to the selected inverse are established in theorem 6.1 below. $\square$

5.1 Low orders and nonlogarithmic specialization

For a single generator $x^\delta P(\log x)$, the first corrections are

$$\begin{aligned} g(y) &= s - \frac{1}{p} s^{1+\delta} P(L) \\ &\quad + \frac{s^{1+2\delta}}{2p^2} (1 + 2\delta + p + \partial_L) P(L)^2 \\ &\quad - \frac{s^{1+3\delta}}{6p^3} (1 + 3\delta + p + \partial_L)(1 + 3\delta + 2p + \partial_L) P(L)^3 + \dots \end{aligned} \quad (5.5)$$

For two distinct generators, the mixed second-order coefficient is

$$C_{\mathbf{e}_i + \mathbf{e}_j}(L) = \frac{1}{p^2} (1 + \delta_i + \delta_j + p + \partial_L) P_i(L) P_j(L). \quad (5.6)$$

There is no factor $1/2$ here, because $(\mathbf{e}_i + \mathbf{e}_j)! = 1$.

When all P_j are constants a_j , the operators become scalars:

$$\begin{aligned} C_{\mathbf{n}} &= \frac{(-1)^N}{p n!} \left(\frac{1+d}{p} + 1 \right)^{\overline{N-1}} \prod_j a_j^{n_j} \\ &= \frac{1}{1+d} \binom{-(1+d)/p}{N} \frac{N!}{n!} \prod_j a_j^{n_j}. \end{aligned} \quad (5.7)$$

Here $(a)^{\bar{k}} = a(a+1) \cdots (a+k-1)$ denotes a rising factorial, with $(a)^{\bar{0}} = 1$. This is the generalized-power analogue of the classical coefficient formula. The derivative operators in (5.1) are exactly the extra structure needed when logarithms occur.

6 Why the generalized series actually converges

Write $m_j = \deg P_j$ and $M_L = 1 + |L|$. The relevant small quantities are

$$u_j = s^{\delta_j} M_L^{m_j}, \quad \eta(s) = \sum_j u_j. \quad (6.1)$$

Each tends to zero, even though $|L| \rightarrow \infty$.

Theorem 6.1 (Uniformly normalized analytic inversion). *There are constants $B > 0$, $\rho > 0$, and $s_0 > 0$, depending on the fixed model, such that*

$$|C_n(L)| \leq B\rho^{-|n|} M_L^{\sum_j m_j n_j} \quad (L \leq \log s_0). \quad (6.2)$$

The series (5.2) converges absolutely for all sufficiently small positive s and equals the branch in theorem 2.3. A total-degree truncation satisfies, for fixed K ,

$$g(y) - s \sum_{|n| \leq K} s^{n \cdot \delta} C_n(L) = \mathcal{O}(s\eta(s)^{K+1}). \quad (6.3)$$

Proof. Keep L fixed temporarily and write $x = sw$. Introduce independent complex parameters z in

$$w^p \left(1 + \sum_j z_j w^{\delta_j} P_j(L + \log w) \right) - 1 = 0. \quad (6.4)$$

Choose a disk $|w - 1| < r_0 < 1$ small enough that $w^p - 1$ has exactly one zero there, namely the simple zero $w = 1$, and no zero on its boundary. The powers and logarithm use their analytic branches on this disk. Such a disk exists by the nonzero derivative p at 1.

Normalize the parameters by $v_j = z_j M_L^{m_j}$. On the closed disk, $\log w$ is bounded. Since P_j is a fixed polynomial,

$$\sup_{L \leq \log s_0, |w-1|=r_0} \frac{|w^{p+\delta_j} P_j(L + \log w)|}{M_L^{m_j}} < \infty.$$

Also $b = \min_{|w-1|=r_0} |w^p - 1| > 0$. Choose a positive polydisk radius ρ so that the sum of these bounds times ρ is smaller than b . Rouché's theorem now gives exactly one zero of (6.4) inside the disk, for every parameter point in that polydisk, uniformly in L . A zero counted with multiplicity one is simple. Local holomorphic dependence patches to a single holomorphic function because the root in the disk is unique.

Its absolute value is bounded by $1 + r_0$. Cauchy's estimates in a slightly smaller polydisk give a bound $B\rho^{-|n|}$ for its coefficient in v^n . Returning to z multiplies the bound by $M_L^{\sum m_j n_j}$, proving (6.2). The coefficient is precisely $C_n(L)$ from theorem 5.1, by the local parameter identity.

At the actual values $z_j = s^{\delta_j}$, the normalized variables v_j are the small quantities (6.1). For sufficiently small s , each $v_j < \rho/2$, so the multivariate series converges absolutely. Its real root satisfies $w \rightarrow 1$ and $x = sw > 0$, hence is the unique inverse germ already selected. For the tail of total degree $K + 1$, use

$$\sum_{|n|=N} \prod_j u_j^{n_j} \leq \left(\sum_j u_j \right)^N$$

and sum the geometric majorant. This gives $Bs(\eta/\rho)^{K+1}/(1 - \eta/\rho)$, which implies (6.3). $\square$

The theorem is stronger than a purely formal reversion statement, but its scope must be kept clear. The forward model is a *finite exact* sum with fixed positive gaps. A divergent infinite asymptotic input, a flat remainder, or a parameter limit in which a gap tends to zero requires additional arguments. Convergence here is not an assertion about every transseries or every inverse problem.

7 Weighted truncation and a finite-border remainder

Total perturbation degree is often the wrong output order. For example, four copies of a small gap can have smaller total weight than two copies of a larger one. The implementation therefore retains $\mathcal{I}_A$ from (3.2). Its finite *exit border* is

$$\mathcal{B}_A = \{\mathbf{n} + \mathbf{e}_j : \mathbf{n} \in \mathcal{I}_A, (\mathbf{n} + \mathbf{e}_j) \cdot \boldsymbol{\delta} \geq A, 1 \leq j \leq r\}. \quad (7.1)$$

Repeated border indices are removed. This border need not be a minimal antichain; minimality is unnecessary for the estimate.

Theorem 7.1 (Logarithm-aware weighted remainder). *Suppose $r > 0$ and $A > 0$. Define*

$$d_* = \min_{\mathbf{b} \in \mathcal{B}_A} \mathbf{b} \cdot \boldsymbol{\delta}, \quad k_* = \max_{\substack{\mathbf{b} \in \mathcal{B}_A \\ \mathbf{b} \cdot \boldsymbol{\delta} = d_*}} \sum_j b_j m_j. \quad (7.2)$$

Then

$$g(y) = s \sum_{\mathbf{n} \in \mathcal{I}_A} s^{\mathbf{n} \cdot \boldsymbol{\delta}} C_{\mathbf{n}}(L) + O\left(s^{1+d_*} (1 + |L|)^{k_*}\right). \quad (7.3)$$

In the output variable, the remainder power is $(1 + d_*)/p$. The bound is safe even if some omitted coefficients cancel; it need not be sharp after such cancellations.

Proof. Every $\mathbf{n} \notin \mathcal{I}_A$ dominates at least one border index componentwise. Indeed, take any path from $\mathbf{0}$ to $\mathbf{n}$ adding one unit vector at a time, and consider its first exit from $\mathcal{I}_A$. This exit lies in $\mathcal{B}_A$ and is componentwise no larger than $\mathbf{n}$.

Set $v_j = \rho^{-1} s^{\delta_j} M_L^{m_j}$, with ρ from theorem 6.1. Its coefficient bound and the preceding cover imply

$$\begin{aligned} \left| g - s \sum_{\mathbf{n} \in \mathcal{I}_A} s^{\mathbf{n} \cdot \boldsymbol{\delta}} C_{\mathbf{n}}(L) \right| &\leq B s \sum_{\mathbf{n} \notin \mathcal{I}_A} v^{\mathbf{n}} \\ &\leq B s \sum_{\mathbf{b} \in \mathcal{B}_A} v^{\mathbf{b}} \prod_{j=1}^r \frac{1}{1 - v_j}. \end{aligned}$$

For small s the product of denominators is bounded. The remaining border sum is finite. Among its terms with smallest power d_* , the largest logarithmic degree is k_* . Every term with larger power is bounded by a constant times $s^{d_*} M_L^{k_*}$ by theorem 2.2. This proves the result. $\square$

7.1 Strict cutoff semantics

To retain inverse powers y^λ with $\lambda < q$, set

$$A = pq - 1. \quad (7.4)$$

Then $(1 + d)/p < q$ is equivalent to $d < A$. The cutoff must satisfy $q > 1/p$ so that the leading block is present. The first possible omitted power can be strictly larger than q if there is a gap in the support. The package reports the border power rather than degrading it to q .

The theorem also explains why appending $O[y]^q$ to an ordinary expression is inappropriate in general. A block of power q multiplied by an unbounded logarithmic polynomial is not $O(y^q)$. The output association has separate fields for its power, logarithmic degree, and scale. The factor $1 + |\log s|$ makes the bound nonnegative without relying on the sign of $\log s$.

7.2 Complexity and exact ordering

The index count is at most

$$\prod_{j=1}^r \left(1 + \left\lfloor \frac{A}{\delta_j} \right\rfloor\right),$$

although most indices in this box are not retained. Each coefficient is formed from a finite polynomial and $N - 1$ first-order differential operators. The exit border has at most $r|\mathcal{I}_A|$ entries before deduplication. Collisions can greatly reduce the number of final blocks.

The package enumerates using exact comparisons, not floating floors of approximate radical quotients. Exact algebraic constants are the primary intended exponents. Other exact numeric real constants may be accepted when every required comparison is proved by the kernel; an unresolved comparison produces a failure, never a guessed ordering. Symbolic exponents are deliberately excluded, because the support order and number of retained indices can change with the parameter.

8 A second engine: Newton inversion in finite jets

The coefficient formula is nonrecursive and particularly transparent for small support sets. A second construction is useful both computationally and as an independent algebraic check. It works entirely in the finite quotient obtained by discarding weights at least A .

Put $g(y) = s(1 + U)$, where U has positive weight. The normalized inverse equation is

$$G(U) = (1 + U)^p + \sum_j s^{\delta_j} (1 + U)^{p+\delta_j} P_j(L + \log(1 + U)) - 1 = 0. \quad (8.1)$$

Its derivative with respect to U is

$$G'(U) = p(1 + U)^{p-1} + \sum_j s^{\delta_j} (1 + U)^{p+\delta_j-1} ((p + \delta_j)P_j + P'_j)(L + \log(1 + U)). \quad (8.2)$$

Its leading coefficient is the nonzero scalar p , so it is a unit.

Theorem 8.1 (Finite Newton reversion). *Starting with $U_0 = 0$, perform*

$$U_{k+1} = U_k - \frac{G(U_k)}{G'(U_k)}, \quad (8.3)$$

truncating all intermediate jets at weight A . The iteration terminates with the unique positive-order solution of (8.1) in this quotient. It equals the weighted truncation of [theorem 5.1](#). If the current error has valuation at least $v > 0$, the next error has valuation at least $2v$.

Proof. Let $\delta_{\min}$ be the smallest positive generator. Existence of a formal solution follows either from the coefficient formula, or coefficient by coefficient: the unknown coefficient of weight d enters $G(U)$ as p times that coefficient, while every other contribution involves only smaller weights. The finite ordering of weights below A makes this induction legitimate. It also proves uniqueness.

Let U_* be that solution and $E = U - U_*$. Formal Taylor expansion, valid because E has positive valuation, gives

$$G(U) = G'(U)E + \mathcal{O}_{\text{formal}}(E^2).$$

Here $\mathcal{O}_{\text{formal}}(E^2)$ denotes membership in the ideal of valuation at least $2 \text{ val } E$, not an analytic big- O assertion in s . Dividing by the unit $G'(U)$ preserves valuation and proves doubling. Initially $\text{val } U_* \geq \delta_{\min}$, unless the solution is identically zero. Thus after finitely many steps the error has valuation at least A and vanishes in the quotient. The exact coefficient construction satisfies the same equation, so uniqueness identifies the two answers. If there are no generators, $U_* = 0$ from the outset. $\square$

For a sparse-jet implementation one need not manipulate expressions containing unevaluated logarithms of series. The following recurrence is more reliable.

Lemma 8.2 (Composition coefficients). *For a scalar a and a polynomial P , define*

$$Q_0(L) = P(L), \quad Q_{k+1}(L) = \frac{(a - k)Q_k(L) + Q'_k(L)}{k + 1}.$$

Then, formally in t ,

$$(1 + t)^a P(L + \log(1 + t)) = \sum_{k \geq 0} Q_k(L) t^k. \quad (8.4)$$

Proof. Let $H(t, L)$ denote the left side. Direct differentiation gives $(1 + t)\partial_t H = aH + \partial_L H$. Comparing the coefficient of t^k gives $(k + 1)Q_{k+1} + kQ_k = aQ_k + Q'_k$, which is the recurrence. The constant coefficient is $P(L)$, so all coefficients agree. $\square$

The algorithm stores a jet as a sorted list of pairs $\{d, Q_d(L)\}$. Addition merges equal weights; multiplication adds weights and multiplies polynomials; reciprocal uses a finite geometric series after dividing by the constant coefficient. For composition, [theorem 8.2](#) generates Q_k while powers of U are multiplied until none remain below the current cutoff. Every operation terminates.

A mathematical operation count for Newton need not translate directly to a speedup in this research implementation. The code uses simple sparse lists and repeated exact simplification, prioritizing auditability over large-order performance. No performance comparison between the two Wolfram implementations was run. A production implementation could cache polynomial products, use a canonical number-field representation for algebraic weights, and increase the cutoff progressively during Newton iteration instead of computing at full precision from the start.

9 Residuals, actual inverse errors, and certification

A correct coefficient calculation must cancel the forward equation, but one must still relate that cancellation to the inverse function. The correct scaling depends on p .

Theorem 9.1 (Residual transport near a monomial core). *Let f satisfy (2.1), let g be its positive inverse, and let $a(y)/s \rightarrow 1$. If*

$$f(a(y)) - y = O(s^{p+v}(1 + |\log s|)^k),$$

then

$$a(y) - g(y) = O(s^{1+v}(1 + |\log s|)^k). \quad (9.1)$$

More generally, an absolute forward residual $R(y)$ gives inverse error $O(s^{1-p}|R(y)|)$ whenever the indicated estimate holds on the segment joining $a(y)$ and $g(y)$.

Proof. Both endpoints are asymptotic to s , so that segment lies in $[s/2, 2s]$ for sufficiently small y . The derivative formula in the proof of [theorem 2.3](#) gives constants $b_1, b_2 > 0$ such that

$$b_1 s^{p-1} \leq f'(x) \leq b_2 s^{p-1} \quad (s/2 \leq x \leq 2s).$$

The mean-value theorem gives $|a - g| \leq |f(a) - f(g)|/(b_1 s^{p-1})$, proving the claims. $\square$

For a requested inverse cutoff q , the normalized cutoff is $A = pq - 1$. The corresponding forward residual cutoff in powers of y is

$$r = 1 + \frac{A}{p} = q + \frac{p-1}{p}. \quad (9.2)$$

Thus “residual zero below q ” is not the correct generic test when $p \neq 1$. The package computes the normalized residual $f(s(1+U))/(cs^p) - 1$ below weight A , and reports its corresponding cutoff (9.2).

An asymptotic scale is not an explicit numerical error bar. For a specified y , a certificate needs an actual neighborhood, derivative bound, and residual bound. The following elementary result states exactly what is required.

Theorem 9.2 (Two-sided residual certificate). *Suppose f is continuous on $[a-h, a+h]$, differentiable in its interior, and $f'(x) \geq m > 0$ there. If $|f(a) - y| \leq mh$, there is a unique solution g of $f(g) = y$ in the interval, and*

$$|g - a| \leq \frac{|f(a) - y|}{m}.$$

Proof. The mean-value theorem gives $f(a-h) \leq f(a) - mh \leq y$ and $f(a+h) \geq f(a) + mh \geq y$. Continuity gives a root between the endpoints, and the positive derivative gives uniqueness. Applying the same derivative bound between a and that root proves the error estimate. $\square$

The package does not search for an interval satisfying these hypotheses or perform outward-rounded interval arithmetic. Accordingly, its field “NumericalIntervalCertificate” is False. Its “FormalResidualZero” field means exact cancellation in the finite jet algebra of the supplied model. These are deliberately different notions. The branch and asymptotic remainder theorems justify the symbolic output; a numerical certificate at a particular argument remains an additional step.

10 Inverting a finite forward expansion with a remainder

One often knows a finite asymptotic model rather than an exact finite function. This can be handled without pretending that the omitted input terms are zero.

Theorem 10.1 (Stability under a differentiated remainder). *Let f_0 be a finite model (2.1), and suppose the actual function is $F(x) = f_0(x) + r(x)$, with*

$$r(x) = O(x^\beta(1 + |\log x|)^k), \quad r'(x) = O(x^{\beta-1}(1 + |\log x|)^k), \quad \beta > p, \quad (10.1)$$

where k is a nonnegative integer. Then F has a positive inverse germ G , and

$$G(y) - f_0^{-1}(y) = O(s^{\beta-p+1}(1 + |\log s|)^k). \quad (10.2)$$

Consequently, coefficients of inverse powers strictly below

$$q_{\max} = \frac{\beta - p + 1}{p} \quad (10.3)$$

are determined by the finite model. A remainder for a truncated model inverse must be combined with (10.2).

Proof. The value bound gives $F(x)/(cx^p) \rightarrow 1$, and the derivative bound is $o(x^{p-1})$. Thus $F'(x) \sim cpx^{p-1} > 0$ and the same branch argument as before gives $G(y)/s \rightarrow 1$. At $a(y) = f_0^{-1}(y) \sim s$ the residual $F(a) - y$ equals $r(a)$, which has size $O(s^\beta(1 + |\log s|)^k)$. Apply the derivative estimate for F and theorem 9.1. This proves (10.2) and then (10.3). $\square$

The theorem does not say that every coefficient at the boundary power is undetermined in every particular problem. It gives a uniform safe rule: all complete blocks strictly below the boundary are determined, while the unknown remainder can affect the boundary block.

The derivative hypothesis cannot simply be omitted. The function

$$F(x) = x + x^2 \sin(x^{-3})$$

satisfies $F(x) = x + O(x^2)$ but

$$F'(x) = 1 + 2x \sin(x^{-3}) - 3x^{-2} \cos(x^{-3})$$

changes sign arbitrarily close to zero. The value approximation alone therefore does not imply a single monotone inverse branch on a full punctured neighborhood. Other hypotheses can replace the derivative bound, for example a separate proved monotonicity and lower-slope condition; they are not inferred automatically here.

The option

```
"InputRemainder" → {beta, k}
```

asserts both bounds in (10.1). The package checks $\beta > p$ and rejects a requested cutoff $q > q_{\max}$. It does *not* prove those input bounds. When combining errors, the smaller power wins; at equal powers the larger logarithmic degree wins. For example,

```
RealInverseSeries[x + x^2 (1 + Log[x]), {x, y}, 4,
  "InputRemainder" → {4, 0}]
```

has a model truncation error $O(y^4(1 + |\log y|)^3)$ and an input-induced error $O(y^4)$. The former dominates, so the reported logarithmic degree is 3. Requesting cutoff 5 from the same remainder information is rejected rather than silently treating an unknown fourth-order forward term as zero.

11 The logarithmic example to arbitrary order

For $f(x) = x + x^2(1 + \log x)$ we have $p = c = \delta = 1$ and $P(L) = 1 + L$. The all-order answer is

$$g(y) = y + \sum_{n \geq 1} y^{n+1} A_n(\log y), \quad A_n(L) = \frac{(-1)^n}{n!} \prod_{k=1}^{n-1} (n + k + 1 + \partial_L)(1 + L)^n. \quad (11.1)$$

This is both a convergent generalized series for sufficiently small y and a power–logarithmic asymptotic expansion to arbitrary order. Keeping $n = 1, \dots, N$ gives

$$g(y) = y + \sum_{n=1}^N y^{n+1} A_n(\log y) + O(y^{N+2}(1 + |\log y|)^{N+1}). \quad (11.2)$$

The first five coefficient polynomials are

$$\begin{aligned} A_1(L) &= -L - 1, \\ A_2(L) &= 2L^2 + 5L + 3, \\ A_3(L) &= -5L^3 - \frac{41}{2}L^2 - 27L - \frac{23}{2}, \\ A_4(L) &= 14L^4 + \frac{241}{3}L^3 + \frac{335}{2}L^2 + 151L + \frac{299}{6}, \\ A_5(L) &= -42L^5 - \frac{3727}{12}L^4 - \frac{5365}{6}L^3 - 1256L^2 - \frac{5177}{6}L - \frac{2789}{12}. \end{aligned} \quad (11.3)$$

Proposition 11.1 (Catalan diagonal and a common factor). *The coefficient of L^n in A_n is*

$$(-1)^n \text{Cat}_n = \frac{(-1)^n}{n+1} \binom{2n}{n}.$$

Every A_n is divisible by $1 + L$.

Proof. Differentiation lowers polynomial degree, so only the scalar parts of all operators contribute to L^n . Their product is $(n+2)(n+3)\cdots(2n) = (2n)!/(n+1)!$. Multiplication by $(-1)^n/n!$ gives the first formula. There are at most $n-1$ derivatives applied to $(1+L)^n$. Each derivative leaves at least one factor $1+L$, proving the second assertion. $\square$

The common factor is also consistent with the exact fixed point: $f(e^{-1}) = e^{-1}$. This is a structural consistency check, not a proof that the displayed series converges at $y = e^{-1}$; the local convergence theorem only guarantees some sufficiently small interval.

The Catalan diagonal makes the logarithmic factor in (11.2) sharp for each fixed N . Indeed, the first omitted block has nonzero leading coefficient $(-1)^{N+1} \text{Cat}_{N+1}$, and every later block is smaller. Thus

$$\frac{|g(y) - g_N(y)|}{y^{N+2} |\log y|^{N+1}} \longrightarrow \text{Cat}_{N+1} \quad (y \rightarrow 0^+).$$

This limit is a useful numerical diagnostic that tests more than just small absolute error.

12 The irrational-power example and its convergence radius

For the entire family $f(x) = x + x^\alpha$, $\alpha > 1$, put $\delta = \alpha - 1$. The answer becomes

$$g(y) = \sum_{n \geq 0} b_n y^{1+n\delta}, \quad b_0 = 1, \quad b_n = \frac{(-1)^n}{n!} (\alpha n)^{n-1} \quad (n \geq 1). \quad (12.1)$$

Equivalently,

$$b_n = (-1)^n \frac{\Gamma(\alpha n + 1)}{\Gamma(n+1)\Gamma(\delta n + 2)}. \quad (12.2)$$

The notation $(a)^k = a(a-1)\cdots(a-k+1)$ denotes a falling factorial, with the empty product equal to 1. Here the gamma quotient is merely a convenient rewrite of a finite product; computing coefficients needs neither gamma evaluation nor a special-function inverse.

Proposition 12.1 (Radius in the natural small variable). *The power series $g(y)/y = \sum_{n \geq 0} b_n z^n$ in $z = y^\delta$ has radius*

$$R_\alpha = \frac{(\alpha-1)^{\alpha-1}}{\alpha^\alpha}. \quad (12.3)$$

In particular, the generalized series converges for positive y with $y^{\alpha-1} < R_\alpha$ and there equals the positive inverse.

Proof. The coefficient formula is (5.1) with $p = 1$, $P = 1$ and $d = n\delta$. Its scalar product is $(\alpha n)(\alpha n - 1)\cdots(\alpha n - n + 2)$, giving (12.1). Applying Stirling's formula [7] to the three gamma factors in (12.2) gives

$$|b_n| \sim \frac{\sqrt{\alpha}}{\sqrt{2\pi} \delta^{3/2}} n^{-3/2} \left(\frac{\alpha^\alpha}{\delta^\delta} \right)^n.$$

The root test proves the radius. On a small positive interval the sum is already the inverse by theorem 6.1. To continue along $0 < z < R_\alpha$, its germ solves $w + zw^\alpha = 1$ with $w = g(y)/y$. This equation has exactly one positive solution and nonzero derivative $1 + \alpha zw^{\alpha-1}$. Along any compact positive subinterval

that solution stays bounded away from zero and continues analytically by the implicit function theorem. Uniqueness of analytic continuation identifies it with the convergent series throughout the indicated interval. $\square$

For $\alpha = \sqrt{2}$, the first terms are

$$g(y) = y - y^{\sqrt{2}} + \sqrt{2} y^{2\sqrt{2}-1} - \frac{6 - \sqrt{2}}{2} y^{3\sqrt{2}-2} + \frac{(4\sqrt{2})(4\sqrt{2}-1)(4\sqrt{2}-2)}{24} y^{4\sqrt{2}-3} + \dots \quad (12.4)$$

Truncating immediately before the fourth correction yields exactly the order requested in the question. The exponents are not rounded to a rational grid. Since the coefficients are constants, the remainder has logarithmic degree zero.

The elementary check $\alpha = 2$ recovers $g(y) = (\sqrt{1+4y} - 1)/2$ and the alternating Catalan coefficients. Its ordinary Taylor radius is $1/4$, consistent with (12.3). This is a useful limit case for both engines.

13 Mixed generators, collisions, and a nonunit core

13.1 A genuinely mixed scale

Consider

$$f(x) = x + x^{\sqrt{2}} + x^2(1 + \log x), \quad \delta = \sqrt{2} - 1.$$

At inverse cutoff $q = 3$, the normalized cutoff is $A = 2$. The retained multi-indices (n, m) satisfy $n\delta + m < 2$. There are eight of them, listed here in increasing weight:

$(0, 0)$	$(1, 0)$	$(2, 0)$	$(0, 1)$	$(3, 0)$	$(1, 1)$	$(4, 0)$	$(2, 1)$
0	δ	2δ	1	3δ	$\delta + 1$	4δ	$2\delta + 1$

The full answer below that cutoff is

$$g(y) = y - y^{\sqrt{2}} + \sqrt{2} y^{2\sqrt{2}-1} - (1 + L)y^2 - \frac{6 - \sqrt{2}}{2} y^{3\sqrt{2}-2} + ((\sqrt{2} + 2)(1 + L) + 1)y^{\sqrt{2}+1} + b_4 y^{4\sqrt{2}-3} - \frac{1}{2} \left[(2\sqrt{2} + 1)(2\sqrt{2} + 2)(1 + L) + 4\sqrt{2} + 3 \right] y^{2\sqrt{2}} + O(y^3(1 + |L|)^2), \quad L = \log y, \quad (13.1)$$

where b_4 is the positive coefficient in (12.4). The final displayed polynomial follows by applying two operators to $1 + L$ with $n = 2, m = 1$. The border weight is 2, from $(0, 2)$, and its logarithmic degree is 2. In contrast, a total-degree cutoff would incorrectly omit some of the pure irrational-power terms that are larger than retained mixed terms.

13.2 Coincident weights must be added

For

$$f(x) = x + x^{3/2} + x^2,$$

the generators are $1/2$ and 1 . The weight 1 receives two contributions: $3/2$ from the multi-index $(2, 0)$ and -1 from $(0, 1)$. Therefore

$$g(y) = y - y^{3/2} + \frac{1}{2}y^2 + O(y^{5/2}).$$

Keeping the two contributions as separate “ordered sectors” would not be a canonical expansion. The package groups them before presenting blocks, and both algorithms pass this collision check in the independent verification.

13.3 The core coefficient and exponent both matter

For

$$f(x) = 3x^2(1 + x^{1/2}(1 + \log x) + 2x)$$

put $s = (y/3)^{1/2}$ and $L = \log s$. Then

$$g(y) = s - \frac{1}{2}s^{3/2}(1 + L) + s^2 \left[\frac{1}{2}(1 + L)^2 + \frac{1}{4}(1 + L) - 1 \right] + O(s^{5/2}(1 + |L|)^3). \quad (13.2)$$

The first contribution at s^2 comes from two copies of the half-power generator, while $-s^2$ comes from the $2x$ generator. The powers of p in the master formula are essential. Treating this as a near-identity problem in y without changing the derivative operator would give incorrect coefficients and an incorrect residual cutoff.

14 The Wolfram Language package

14.1 Loading and the principal function

Unpack the archive and load the package by its actual filesystem path:

```
Get["/path/to/real-inverse-series/Kernel/RealInverseSeries.wl"];
Clear[x, y];
r = RealInverseSeries[x + x^2 (1 + Log[x]), {x, y}, 5];
r["Expansion"]
r["Remainder"]
```

The third argument is a *strict power cutoff*, not a number of terms and not a perturbation order. A request with cutoff 5 includes the complete blocks of powers 1, 2, 3, 4. For the irrational example the call reproducing the question’s four displayed terms is

```
RealInverseSeries[x + x^Sqrt[2], {x, y}, 4 Sqrt[2] - 3]
```

All input and output variable symbols must be unassigned and distinct. The forward expression must be independent of the output symbol: it cannot simultaneously be used as an input parameter.

14.2 Accepted input and deliberate failures

The parser expands a finite sum of terms $x^a P(\log x)$, collects equal powers, and proves the conditions on the smallest remaining power. Its leading coefficient must be a positive scalar, not a polynomial in the logarithm. Every power and the cutoff must be exact, finite, numeric real constants whose required comparisons can be established exactly. Rational numbers and exact algebraic numbers are the primary use cases. Constants such as π may also be accepted when all comparisons are decidable by exact simplification. They are not converted to floating-point numbers.

Polynomial coefficients may contain symbolic parameters if their reality is proved using `Assumptions`. The leading coefficient must likewise be proved positive. For example,

```
RealInverseSeries[x + a x^(3/2), {x, y}, 5/2,
  Assumptions → Element[a, Reals]]
```

returns the polynomial $y - ay^{3/2} + \frac{3}{2}a^2y^2$ with a conditional remainder. No uniformity in unbounded parameter sets is asserted.

Inputs such as $x + x^{1.5}$, a logarithmic leading core, a symbolic exponent, or $x + \text{Exp}[-1/x]$ return **Failure** rather than an unsupported expansion. Inexact coefficients are also rejected: exact symbolic residual cancellation is part of the contract. A user may apply **N** to the returned expansion for numerical evaluation after the symbolic construction.

14.3 Options

Option and default	Meaning
Method -> "Lagrange"	Use the closed multi-index formula. "Newton" selects the independent sparse-jet Newton engine.
Assumptions -> True	Conditions used to prove coefficient reality, core positivity, equality, and exact comparison. They are assumptions, not numerical evidence.
"InputRemainder" -> None	Treat the supplied finite model as exact. A pair {beta,k} instead asserts the two differentiated bounds in (10.1).
"MaxIndices" -> 10000	Limit the enumerated order ideal. Exceeding the limit returns a failure; no partial expansion is presented as complete.
"VerifyResidual" -> True	Recompose the output in finite jets and require exact cancellation below the appropriate residual cutoff. False skips that check and marks the field as not computed.

14.4 Result fields and representation

The principal return value is an association, not a bare expression. The most useful fields are as follows.

Field	Meaning
"Expansion"	The finite expression, with positive-real powers of y/c and logarithms substituted.
"Remainder"	An association containing the nonnegative error scale, power, logarithmic degree, provenance, and the explicit statement that it is not a numerical interval certificate. Scale 0 denotes an exact inverse of the supplied exact model.
"Blocks"	Sorted pairs $\{\lambda, Q(\ell)\}$ representing $(y/c)^\lambda Q(\log(y/c)/p)$. Equal powers are merged.
"LogVariable", "LogSubstitution"	The private polynomial variable ℓ and its intended substitution. Its generated name is not part of the interface.
"Core", "Generators"	The parsed c, p and the pairs $\{\delta_j, P_j(\ell)\}$.
"Cutoff", "IndexCount"	The requested strict inverse cutoff and the number of retained multi-indices before collisions.
"FormalResidualZero"	Whether the optional exact recomposition check completed successfully.
"VerifiedResidualCutoff"	The forward power cutoff $q + (p - 1)/p$, not generally q .
"Branch", "Conditions"	The positive inverse-germ specification and supplied conditions.
"ForwardRemainderContract"	Whether the finite input is exact or which bounds the user has assumed.

The internal sparse representation is retained for `InverseResidual`; consumers should not edit `"InternalData"`. This release does not define overloaded arithmetic on its result associations. To compose or combine several results, their remainder contracts must also be propagated; manipulating only `"Expansion"` discards that information.

14.5 Residual inspection and generic perturbation inversion

For a result `r`,

```
InverseResidual[r]
InverseResidual[r, 6]
```

returns the formal expansion of the finite model's residual. The first call uses (9.2). The second inspects all residual powers strictly below 6, and can reveal the first nonzero omitted coefficient. It does not manufacture additional inverse coefficients. For the two-block logarithmic approximation,

$$f(y - y^2(1 + \log y)) - y = -y^3(2 \log^2 y + 5 \log y + 3) + O(y^4(1 + |\log y|)^3).$$

The sign is opposite to the next inverse correction, as expected.

The separate utility

```
PerturbativeInverse[phi, h, {x, y, eps}, n]
```

implements (4.1) through degree n in `eps`. Here `phi` is an expression in y for a specified local inverse of a core F_0 , and `h` is the perturbation expressed in x . For example,

```
PerturbativeInverse[y, x^2 (1 + Log[x]), {x, y, eps}, 2]
```

gives

$$y - \varepsilon y^2(1 + \log y) + \varepsilon^2 y^3(2 \log^2 y + 5 \log y + 3).$$

This function is an exact finite differential formula, not an automatic real-branch detector. A caller supplying an arbitrary `phi` must ensure that it is the intended local inverse and satisfies the theorem's hypotheses. Setting `eps -> 1` requires a convergence or asymptotic argument; the principal power-log function supplies that argument only for its supported model class.

14.6 Relationship to built-in series operations

The historical question demonstrates a particular attempt involving `InverseFunction` and `Series`; it is not a benchmark of present kernel behavior. Official documentation describes `InverseSeries` as reversion of a series and describes the rational power-grid structure of `SeriesData`. It also explicitly allows logarithms or slower functions as coefficients in some series, and leaves symbolic powers outside a `SeriesData` object [9, 10]. Therefore it would be incorrect to say that built-in series have no logarithmic support at all.

The reason for the custom representation is narrower: this algorithm needs an arbitrary finitely generated real exponent support, complete polynomial-logarithmic blocks, exact collision handling, and an explicit analytic meaning for the remainder. It avoids reliance on how a particular kernel version chooses to represent such a result. No `PowerExpand` is used, so positive-real branch information is not silently replaced by generally invalid complex identities.

15 Verification record and reproducibility

15.1 What was executed

The distribution contains an independent implementation in `verification/reference.py`. It uses exact SymPy expressions for weights and coefficients, and mpmath for numerical evaluation. Its two engines implement the closed operator formula and Newton jet inversion separately. The program `verification/verify.py` was executed and recorded **79 successful checks** in `verification/verification-report.json`. Its environment was Python 3.13.5, SymPy 1.14.0, and mpmath 1.3.0. These are recorded execution versions, not minimum supported versions or claims about the latest releases.

The symbolic checks include the direct derivative formula through eight orders in the logarithmic example, the Catalan diagonal, explicit low coefficients, irrational-power coefficients through eight orders, exact cutoff boundaries, mixed supports, and resonant collision sums. For eight models, the independent Newton and Lagrange engines agree and the forward residual cancels at the required normalized order. The models include a nonunit core, a fractional core exponent, logarithmic coefficients, negative perturbations, and the exact monomial case. Five additional nonunit-core coefficients were checked directly by differentiating in the original core coordinate.

Numerical checks were performed with 250 decimal digits of working precision. Roots were found by Newton iteration in the normalized variable $w = x/s$, not by subtracting unscaled quantities near zero. The normalized forward residual at each computed root was below 10^{-240} in the recorded checks. A residual reported as zero in the JSON file means zero at that finite working precision, not an exact symbolic root. The numerical calculations use ordinary multiprecision arithmetic, not interval arithmetic.

For the logarithmic example, [table 3](#) gives a small selection of the recorded values. The scaled error is

$$\frac{|g(y) - g_N(y)|}{y^{N+2} |\log y|^{N+1}}, \quad g_N = y + \sum_{n=1}^N y^{n+1} A_n(\log y).$$

The predicted limits are 2 for $N = 1$ and 14 for $N = 3$.

y	corrections N	$ g - g_N $	scaled error
10^{-2}	1	2.420207×10^{-5}	1.141198
10^{-2}	3	1.486663×10^{-7}	3.305444
10^{-4}	1	1.268500×10^{-10}	1.495337
10^{-4}	3	5.096420×10^{-16}	7.082107
10^{-8}	1	5.895398×10^{-22}	1.737408
10^{-8}	3	1.163926×10^{-34}	10.108872

Table 3: Measured inverse errors at 250-digit working precision. The table is generated by the verification program from its recorded numerical values; entries are rounded for display.

To reproduce the independent checks from the distribution root, run

```
python -m pip install -r verification/requirements.txt
python verification/verify.py
```

The second command regenerates the JSON report and the table source. It requires no Wolfram kernel. The word “independent” here means a separate implementation and a second algorithmic route, not an externally reviewed or formally verified program.

15.2 What was not executed

The native file `Tests/RealInverseSeries.wlt` contains **33 Wolfram Language regression tests**. They cover the two question examples, logarithmic remainders, exact irrational cutoffs, mixed weights, collisions, pure and nonunit cores, the two methods, residual inspection, parameter assumptions, forward remainder limits, resource limits, unsupported inputs, and the perturbation utility. These tests were supplied but *not run in a Wolfram kernel* during preparation. Both attempts to access the connected Wolfram service returned an unavailable-endpoint error, and no local kernel was present. The independent Python results therefore must not be read as a native Wolfram test pass.

The native command is

```
wolframscript -file Tests/run-tests.wl
```

or, from a notebook after loading the package,

```
TestReport["/path/to/real-inverse-series/Tests/RealInverseSeries.wlt"]
```

The runner records the actual kernel version and report when executed, and exits unsuccessfully if the report does not succeed. It uses the current documented "ReportSucceeded" property when available, with an older-report count fallback [11]. No native report file is shipped as though such an execution had occurred.

The Wolfram source was manually reviewed and subjected to a delimiter, string, and nested-comment balance check. That check can catch malformed source structure, but it is not a Wolfram parser, evaluator, or proof of semantic correctness. Native execution remains the next validation step for adopting the package in a production workflow. The mathematical proofs do not depend on that execution, but implementation defects are not ruled out merely by proving the algorithm.

15.3 Rebuilding the article

From the article directory, run

```
pdflatex -interaction=nonstopmode -halt-on-error real-inverse-series.tex
pdflatex -interaction=nonstopmode -halt-on-error real-inverse-series.tex
pdflatex -interaction=nonstopmode -halt-on-error real-inverse-series.tex
```

A small `build.sh` script in the distribution root performs these passes. The TeX source uses standard AMS, NewTX, listings, and hyperref packages. It includes the full Wolfram source from the adjacent Kernel directory and the generated numerical table; these paths are relative, so preserve the distribution layout when rebuilding.

16 Boundaries and extensions of the theory

16.1 Finite jets extracted from a more general forward function

A forward expression need not itself be a finite power-log sum for its inverse to be accessible to the theory. For example, an analytic factor can be expanded to a finite Taylor polynomial with a differentiated remainder. Combining that polynomial with finitely many power-log terms produces a model to which [theorem 10.1](#) applies. The truncation order must be selected from the inverse error law, not merely copied from the desired inverse cutoff.

As a concrete example, let

$$F(x) = \sin x + x^2 \log x = x + x^2 \log x - \frac{x^3}{6} + O(x^5).$$

Taylor's theorem also gives the derivative remainder $O(x^4)$. Thus the finite model can be submitted as

```
RealInverseSeries[x + x^2 Log[x] - x^3/6, {x, y}, 5,
  "InputRemainder" → {5, 0}]
```

The main parser does not expand $\text{Sin}[x]$ itself. This explicit separation keeps the origin and order of the forward remainder visible.

16.2 Other centers and the point at infinity

For a right-hand inverse near (x_0, y_0) , first write $t = x - x_0$ and $u = y - y_0$. If the translated positive-side equation is a finite model in t , the package applies to that model, and the result is translated back. The local side and the sign of the leading core must be chosen explicitly; a left-hand branch may need a substitution $t = x_0 - x$ and a corresponding target sign change.

An infinity problem can sometimes be converted to an endpoint problem by $t = 1/x$ and a suitable target reciprocal. This is a mathematical reduction, not an automatic feature of this release. The inverse reconstruction may amplify errors, so a bound on t must be transported through $x = 1/t$. In particular, a relative error in t is preferable to an unsupported absolute big- O statement after reciprocation.

16.3 Logarithmic leading cores and deeper transseries

The input

$$f(x) = -x \log x$$

is not in (2.1): the leading coefficient is not a constant. Its small positive inverse is

$$g(y) = -\frac{y}{W_{-1}(-y)}.$$

Indeed, writing $x = e^u$ gives $ue^u = -y$, and the branch $u \rightarrow -\infty$ selects W_{-1} rather than W_0 . Thus a logarithmic core changes the natural scale and branch analysis; it cannot be accommodated by simply allowing $p = 1$ and a coefficient called “ $-\log x$ ” in the present unit algebra. The generic perturbation utility can operate around a specified Lambert core at a fixed positive point, but it does not supply an automatic all-order endpoint remainder theorem for that larger class.

Negative powers of $\log x$, nested logarithms, exponential sectors, and oscillatory factors likewise require a larger coefficient algebra and new composition estimates. General transseries provide a framework for such extensions; Edgar's introduction gives broader context [8]. The present construction uses only finite generator semigroups and polynomial logarithmic blocks, so its proofs and finite algorithms do not depend on implementing the full transseries field.

A flat correction is particularly instructive. Replacing a finite model by $f(x) + e^{-1/x}$ can leave every algebraic-logarithmic asymptotic coefficient unchanged, yet change the actual inverse by a nonzero beyond-all-orders term. The differentiated-remainder theorem controls it at every fixed finite algebraic order, but an algebraic expansion alone does not identify that exponentially small term. A package claiming the exact finite model's convergent inverse for the perturbed function would be making a false identification.

16.4 Parameter limits and large-order computation

All statements above are for fixed positive p, c, δ_j and fixed polynomials. As $\delta_{\min} \rightarrow 0^+$, the number of indices below a fixed cutoff can diverge and $s^{\delta_{\min}}$ need not be small in a joint limit. A uniform double-scaling theory would need a new small parameter and uniform hypotheses. This is why the implemented parser requires fixed exact exponents instead of attempting symbolic ordering across parameter regions.

For large orders, the most promising engineering extensions are canonical algebraic-number coordinates, memoized coefficient operators, sparse polynomial arrays, a heap-based support enumerator, and precision-doubling Newton truncation. A sharper optional remainder could inspect cancellation in successive omitted blocks. The present border bound deliberately avoids that unbounded search: it always terminates and remains valid even when its first possible omitted block vanishes.

17 Conclusion

The two original examples are not exceptions requiring unrelated tricks. They are instances of a single power–logarithmic inversion theorem. The correct procedure is to select the positive real germ, normalize the monomial core, use an auxiliary parameter at positive points, and only then reorganize the coefficients by their actual asymptotic weights. The logarithmic derivative operator produces every polynomial coefficient; local finiteness makes irrational supports computable; normalized analytic inversion supplies convergence for finite exact models; and a finite exit border supplies a safe logarithmic remainder at arbitrary power cutoffs.

The package implements this pipeline in two ways and separates the finite expression, formal residual, input assumptions, and analytic error scale. For the question’s first example, it also corrects the ordinary-Landau interpretation of the two-term remainder. For the second, it reproduces the irrational exponents and coefficients exactly. The mathematical results are proved here, independent symbolic and numerical checks are recorded, and native Wolfram execution is explicitly left unclaimed.

A Distribution map and source provenance

Path	Contents
article/real-inverse-series.tex	This article’s source.
article/real-inverse-series.pdf	Compiled article.
article/numerical-table.tex	Table generated by the numerical verification.
Kernel/RealInverseSeries.wl	Complete Wolfram package.
Kernel/init.m	Loader for the adjacent package file.
Examples/examples.wl	Reproducible usage examples.
Tests/RealInverseSeries.wlt	33 native regression tests, not executed here.
Tests/run-tests.wl	Native runner that records the kernel version.
verification/reference.py	Independent exact reference implementation.
verification/verify.py	The executed verification program.
verification/verification-report.json	All 79 checks, coefficients, Newton traces, numerical values, and execution versions.
verification/verification-log.txt	Console output of the executed run.
verification/static-wl-check.py	Structural source check, not a kernel parser.
README.md, build.sh, LICENSE	Usage, build instructions, and reuse terms.

The user’s uploaded archive was used as the local question source. The relevant repository companion sections were read in the source blob 05c56ce5ee60788a67f365110b62810550cba817; the group README had blob 516aaa30e84cb0036a00f803cb6e8e3026febb8e. These are content identifiers, not claims that the entire repository was audited. External materials are cited below and are not redistributed as though they were authored for this package.

B Complete Wolfram Language package

The following listing is included directly from the distributed file, so the article and package source cannot silently diverge during a normal rebuild. The complete executable file is preferable to copying code from the PDF, whose line wrapping is for reading only.

```
(* ::Package:: *)
(* RealInverseSeries 1.0.0
   Positive-real power-log inversion at zero.
   Copyright (c) 2026. Distributed under the MIT license.

   This implementation does not use InverseFunction, PowerExpand, or a
   floating-point fallback to order exponents. Its result is an Association,
   not SeriesData: logarithmic error factors must not be hidden inside  $O[y]^q$ .
   See the article and Tests/ for the precise contract and validation status.
*)

BeginPackage["RealInverseSeries"];

RealInverseSeries::usage =
  "RealInverseSeries[f,{x,y},q] constructs the positive-real inverse of f(x) near zero, retaining complete
  power-log blocks with y exponent strictly less than q. The result is an Association with keys Expansion,
  Remainder, Blocks, and FormalResidualZero. Supported inputs are finite sums  $x^a P(\log x)$  with a
  positive monomial leading block  $c x^p$ ,  $c > 0$  and  $p > 0$ . Exponents and the cutoff must be exact real numeric
  constants. Use Assumptions for symbolic real coefficients.";

InverseResidual::usage =
  "InverseResidual[result,r] expands f(result["Expansion"])-y through all y powers strictly below r.
  InverseResidual[result] uses the residual cutoff corresponding to the requested inverse cutoff. The
  returned Association records a formal truncated residual, not a numerical interval certificate. A
  declared input remainder is not included in this residual.";

PerturbativeInverse::usage =
  "PerturbativeInverse[phi,h,{x,y,eps},n] returns the degree-n Taylor polynomial in eps of the branch solving
   $F(x)+\epsilon h(x)=y$ , where phi is a specified local inverse of F expressed in y. It uses  $\phi+\sum_{k=1}^n (-\epsilon)^k D[(h/x \rightarrow \phi)^k D[\phi,y],\{y,k-1\}]/k!,\{k,1,n\}$ . Setting eps=1 requires a separate convergence or
  remainder argument unless the power-log theorem applies.";

$RealInverseSeriesVersion::usage = "Version string for RealInverseSeries.";
$RealInverseSeriesVersion = "1.0.0";

Options[RealInverseSeries] = {
  Assumptions → True, Method → "Lagrange",
  "InputRemainder" → None, "MaxIndices" → 10000,
  "VerifyResidual" → True
};

Begin["Private"];

$failureTag = Unique["RealInverseSeriesFailure$"];
$assumptions = True;
$maxIndices = 10000;

fail[tag_, text_, data_: <|>] :=
  Throw[Failure[tag, Join["MessageTemplate" → text|>, data]], $failureTag];

zeroQ[z_] := TrueQ[z == 0] ||
  TrueQ[FullSimplify[z == 0, Assumptions → $assumptions]];

realQ[z_] := TrueQ[FullSimplify[Element[z, Reals],
  Assumptions → $assumptions]];

exactRealQ[z_] := FreeQ[z, _Real] && NumericQ[z] && realQ[z] &&
  FreeQ[z, Indeterminate | ComplexInfinity | _DirectedInfinity];

ce[z_] := FullSimplify[z, Assumptions → $assumptions];
cp[z_] := Expand[FullSimplify[Expand[z], Assumptions → $assumptions]];

(* No approximate comparison is permitted, including in Sort. *)
cmp[a_, b_] := Module[{d = ce[a - b]},
  If[TrueQ[d == 0], Return[0]];
  If[TrueQ[d < 0], Return[-1]];
```

```

If[TrueQ[d > 0], Return[1]];
If[TrueQ[FullSimplify[d < 0, Assumptions → $assumptions]], Return[-1]];
If[TrueQ[FullSimplify[d > 0, Assumptions → $assumptions]], Return[1]];
fail["UndecidableOrder", "An exponent comparison could not be certified exactly.",
  <|"Left" → a, "Right" → b|>]
];

(* Sparse jets: sorted lists {weight, polynomial in a private log symbol}.
   Canonicalization adds colliding weights before discarding zero blocks. *)
spCanonical[terms_List, cut_] := Module[{s, groups, out},
  s = Select[terms, cmp[#][1], cut] < 0 &];
  If[s === {}, Return[{}]];
  s = Sort[s, cmp[#1][1], #2][1]] < 0 &];
  groups = Split[s, cmp[#1][1], #2][1]] == 0 &];
  out = {ce[#][1, 1]], cp[Total[#][All, 2]]]} & /@ groups;
  Select[out, !zeroQ[#][2]] &]
];

spAdd[a_List, b_List, cut_] := spCanonical[Join[a, b], cut];
spScale[a_List, c_] := ({#[1], Expand[c #[2]]} &) /@ a;

spMul[a_List, b_List, cut_] := Module[{out = {}, w, i, j},
  Do[
    w = ce[a[[i, 1]] + b[[j, 1]]];
    If[cmp[w, cut] < 0,
      AppendTo[out, {w, Expand[a[[i, 2]] b[[j, 2]]}]],
    {i, Length[a]}, {j, Length[b]}];
  spCanonical[out, cut]
];

spReciprocal[a_List, cut_, ell_] := Module[{c, rest, term, out},
  If[a === {} || cmp[a[[1, 1]], 0] != 0,
    fail["NonunitJet", "The jet to be inverted has no nonzero constant block."]];
  c = a[[1, 2]];
  If[zeroQ[c] || !FreeQ[c, ell],
    fail["NonunitJet", "The leading jet coefficient must be a nonzero scalar."]];
  rest = spScale[Rest[a], -1/c];
  term = {{0, 1}}; out = {{0, 1/c}};
  While[term != {} && rest != {},
    term = spMul[term, rest, cut];
    out = spAdd[out, spScale[term, 1/c], cut];
  ]
];

(* terms = {{externalWeight, exponentOf(1+u), P(ell)}, ...}.
   Q[k+1] = ((a-k) Q[k] + D Q[k])/(k+1) is the Taylor
   coefficient of (1+u)^a P(ell+Log[1+u]). *)
spCompose[terms_List, u_List, cut_, ell_] :=
Module[{out = {}, d, a, q, power, piece, k, i},
  Do[
    d = terms[[i, 1]]; a = terms[[i, 2]]; q = terms[[i, 3]];
    If[cmp[d, cut] < 0,
      power = {{0, 1}}; k = 0;
      While[power != {},
        piece = ({ce[d + #[1]], Expand[q #[2]]} &) /@ power;
        out = spAdd[out, piece, cut];
        q = Expand[((a - k) q + D[q, ell])/(k + 1)];
        power = spMul[power, u, ce[cut - d]];
        k++;
      ]
    ], {i, Length[terms]}];
  spCanonical[out, cut]
];

parseModel[f_, x_, ell_] := Module[
  {expanded, terms, parsed = {}, facs, a, q, fac, i, j, blocks, p, c, deltas, polys},
  If[!FreeQ[f, _Real],
    fail["InexactInput", "Use exact coefficients and exponents; machine-real input is not accepted."]];
  expanded = Expand[f /. Log[x] → ell];
  terms = If[Head[expanded] === Plus, List @@ expanded, {expanded}];
  Do[
    facs = If[Head[terms[[i]]] === Times, List @@ terms[[i]], {terms[[i]]}];
    a = 0; q = 1;

```

```

Do[
  fac = facs[[j]];
  Which[
    fac == x, a += 1,
    Head[fac] == Power && fac[[1]] == x && FreeQ[fac[[2]], x],
    a += fac[[2]],
    FreeQ[fac, x], q *= fac,
    True, fail["UnsupportedInput",
      "Input must be a finite sum x^a P(Log[x]); normalize other functions first.",
      <|"UnsupportedFactor" → fac|>]
  ], {j, Length[facs]};
a = ce[a]; q = Expand[q];
If[!exactRealQ[a],
  fail["Exponent", "Every exponent must be an exact real numeric constant.",
    <|"Exponent" → a|>]];
If[!PolynomialQ[q, ell],
  fail["LogPolynomial", "Only nonnegative integer powers of Log[x] are supported."]];
If[!(And @@ (realQ /@ CoefficientList[q, ell])),
  fail["CoefficientReality", "Coefficient reality was not proved. Supply suitable Assumptions.",
    <|"CoefficientPolynomial" → q|>]];
AppendTo[parsed, {a, q}], {i, Length[terms]};
blocks = spCanonical[parsed, Infinity];
If[blocks == {}, fail["ZeroFunction", "The zero function has no inverse germ."]];
p = blocks[[1, 1]]; c = blocks[[1, 2]];
If[cmp[p, 0] <= 0,
  fail["CoreExponent", "The leading exponent p must be positive for the zero-to-zero branch."]];
If[!FreeQ[c, ell],
  fail["LogarithmicCore", "The leading block must be c x^p, not a logarithmic core."]];
If[!TrueQ[FullSimplify[c > 0, Assumptions → $assumptions]],
  fail["CoreCoefficient", "The leading coefficient c must be provably positive."]];
deltas = (ce[#[[1]] - p] &) /@ Rest[blocks];
polys = (cp[#[[2]]/c] &) /@ Rest[blocks];
{p, c, deltas, polys}
];

(* Enumerate the finite down-set n.delta < A, without rounding a bound.
  Resource failure is explicit rather than a silently incomplete expansion. *)
indicesBelow[deltas_List, A_] := Module[{visit, out = {}, r = Length[deltas]},
  visit[j_, prefix_, w_] := Module[{k = 0, ww},
    If[j > r,
      AppendTo[out, {prefix, w}];
      If[Length[out] > $maxIndices,
        fail["IndexLimit", "The requested cutoff exceeds MaxIndices; no partial series is returned."]];
      Return[Null];
      ww = w;
      While[cmp[ww, A] < 0,
        visit[j + 1, Append[prefix, k], ww];
        k++; ww = ce[w + k deltas[[j]]]
      ]
    ];
  visit[1, {}, 0]; out
];

borderOf[indices_List, deltas_List, A_] := Module[{out = {}, nn, w, i, j},
  Do[
    w = ce[indices[[i, 2]] + deltas[[j]]];
    If[cmp[w, A] >= 0,
      nn = indices[[i, 1]]; nn[[j]] += 1;
      AppendTo[out, {nn, w}];
      {i, Length[indices]}, {j, Length[deltas]};
    DeleteDuplicates[out]
  ];
];

inverseCoefficient[ns_List, d_, p_, polys_List, ell_] :=
Module[{n = Total[ns], q, k},
  If[n == 0, Return[1]];
  q = Times @@ MapThread[Power, {polys, ns}];
  Do[q = Expand[(1 + d + p k) q + D[q, ell]], {k, 1, n - 1}];
  cp[(-1)^n q/(p^n (Times @@ (Factorial /@ ns)))]
];

lagrangeUnit[indices_List, p_, polys_List, A_, ell_] :=
spCanonical[{#[[2]], inverseCoefficient[#[[1]], #[[2]], p, polys, ell]} &) /@ indices, A];

```

```

modelTerms[p_, deltas_List, polys_List] :=
  Prepend[MapThread[#, {p + #1, #2} &, {deltas, polys}], {0, p, 1}];

newtonUnit[p_, deltas_List, polys_List, A_, ell_, iterationLimit_] := Module[
  {terms, derivativeTerms, u = {}, r, derivative, correction, step = 0},
  terms = modelTerms[p, deltas, polys];
  derivativeTerms = ({#[[1]], #[[2]] - 1,
    Expand[#[[2]] #[[3]] + D[#[[3]], ell]]} &) /@ terms;
  While[True,
    r = spAdd[spCompose[terms, u, A, ell], {{0, -1}}, A];
    If[r === {}, Return[spAdd[u, {{0, 1}}, A]]];
    If[step >= iterationLimit,
      fail["NewtonLimit", "Newton jets did not stabilize within the internal iteration bound."];
    derivative = spCompose[derivativeTerms, u, A, ell];
    correction = spMul[r, spReciprocal[derivative, A, ell], A];
    u = spAdd[u, spScale[correction, -1], A]; step++
  ]
];

RealInverseSeries[f_, {x_Symbol, y_Symbol}, cutoff_, OptionsPattern[]] :=
  Catch[Block[{assumptions = OptionValue[Assumptions],
    $maxIndices = OptionValue["MaxIndices"]},
    Module[{ell = Unique["ell$"], p, c, deltas, polys, A, indices, border,
      unit, method = OptionValue[Method], inputRemainder = OptionValue["InputRemainder"],
      verify = OptionValue["VerifyResidual"], beta, inputDegree, inputGap,
      ds = Infinity, kd = 0, degrees, candidates, remainderPower, remainderScale,
      residual = Missing["NotComputed"], expansion, logSub, remainderSource = "ModelTruncation"},
      If[x === y, fail["Variables", "Use distinct unassigned input and output symbols."]];
      If[!FreeQ[f, y], fail["Variables", "The forward expression must be independent of the output symbol."]];
      If[!IntegerQ[$maxIndices] || $maxIndices < 1,
        fail["MaxIndices", "MaxIndices must be a positive integer."]];
      If[!MemberQ[{True, False}, verify],
        fail["VerifyResidual", "VerifyResidual must be True or False."]];
      If[!MemberQ[{"Lagrange", "Newton"}, method],
        fail["Method", "Method must be Lagrange or Newton."]];
      If[!exactRealQ[cutoff],
        fail["Cutoff", "The cutoff must be an exact finite real numeric constant."]];
      {p, c, deltas, polys} = parseModel[f, x, ell];
      A = ce[p cutoff - 1];
      If[cmp[A, 0] <= 0,
        fail["Cutoff", "The cutoff must exceed the leading inverse exponent 1/p."]];
      If[inputRemainder != None,
        If[!MatchQ[inputRemainder, {_, _Integer}],
          fail["InputRemainder", "InputRemainder must be None or {beta,k} with integer k>=0."]];
        {beta, inputDegree} = inputRemainder;
        If[!exactRealQ[beta] || inputDegree < 0,
          fail["InputRemainder", "Input remainder data must have exact real beta and integer k>=0."]];
        inputGap = ce[beta - p];
        If[cmp[inputGap, 0] <= 0,
          fail["InputRemainder", "The input remainder exponent beta must exceed p."]];
        If[cmp[A, inputGap] > 0,
          fail["InsufficientInputOrder", "The requested inverse cutoff exceeds the order justified by the input remainder."],
          <|"MaximumInverseCutoff" -> ce[(beta - p + 1)/p]|>]]];
      indices = indicesBelow[deltas, A];
      border = borderOf[indices, deltas, A];
      unit = If[method === "Lagrange",
        lagrangeUnit[indices, p, polys, A, ell],
        newtonUnit[p, deltas, polys, A, ell, Length[indices] + 2]];
      If[border != {},
        ds = First[Sort[border[[All, 2]], cmp[#1, #2] < 0 &]];
        degrees = Exponent[#, ell] & /@ polys;
        candidates = Select[border, cmp[#[[2]], ds] == 0 &];
        kd = Max[(Total[MapThread[Times, {#[[1]], degrees}]] &) /@ candidates];
      ];
      If[inputRemainder != None,
        Which[
          ds === Infinity || cmp[inputGap, ds] < 0,
            ds = inputGap; kd = inputDegree; remainderSource = "InputRemainder",
          cmp[inputGap, ds] == 0,
            kd = Max[kd, inputDegree]; remainderSource = "ModelAndInputRemainder",

```

```

    True, remainderSource = "ModelTruncation"
  ]
];
logSub = ell → Log[y/c]/p;
expansion = Total[(y/c)^ce[(1 + #[[1]])/p] (#[[2]] /. logSub) & /@ unit];
If[ds === Infinity, remainderSource = "ExactInverse"];
remainderPower = If[ds === Infinity, Infinity, ce[(1 + ds)/p]];
remainderScale = If[ds === Infinity, 0,
  (y/c)^remainderPower (1 + Abs[Log[y/c]/p])^kd];
If[TrueQ[verify],
  residual = spAdd[
    spCompose[modelTerms[p, deltas, polys], spAdd[unit, {{0, -1}}, A], A, ell],
    {{0, -1}}, A];
If[residual != {},
  fail["InternalResidual", "The computed inverse failed the exact truncated residual check.",
    <|"ResidualBlocks" → residual|>]];
];
<|"Expansion" → expansion,
  "Remainder" → <|"Scale" → remainderScale, "Power" → remainderPower,
    "LogDegree" → kd, "Source" → remainderSource,
    "Meaning" → "Inverse minus Expansion is  $O(\text{Scale})$  as  $y \rightarrow 0+$ ; 0 means an exact inverse of the supplied
    exact model.",
    "NumericalIntervalCertificate" → False|>,
  "Blocks" → ({ce[(1 + #[[1]])/p], #[[2]]} &) /@ unit,
  "LogVariable" → ell, "LogSubstitution" → logSub,
  "Core" → <|"Coefficient" → c, "Exponent" → p|>,
  "Generators" → MapThread[{#1, #2} &, {deltas, polys}],
  "Cutoff" → cutoff, "NormalizedCutoff" → A,
  "Branch" → "Positive-real inverse germ with  $x/(y/c)^{(1/p)} \rightarrow 1$  as  $y \rightarrow 0+$ ",
  "Conditions" → ($assumptions && y > 0),
  "Method" → method, "IndexCount" → Length[indices],
  "FormalResidualZero" → If[verify, True, Missing["NotComputed"]],
  "VerifiedResidualCutoff" → If[verify, ce[1 + A/p], Missing["NotComputed"]],
  "ForwardRemainder" → inputRemainder,
  "ForwardRemainderContract" → If[inputRemainder === None, "Exact finite input",
    "User assumes  $r=O(x^\beta (1+|\log|x||)^\alpha)$  and  $r'=O(x^{\beta-1} (1+|\log|x||)^\alpha)$ . These bounds are not
    checked by the package."],
  "Input" → f, "InputVariable" → x, "OutputVariable" → y,
  "Assumptions" → $assumptions,
  "InternalData" → {p, c, ell, deltas, polys, unit},
  "Version" → $RealInverseSeriesVersion|>
]], $failureTag];

RealInverseSeries[___] := Failure["Arguments", <|
  "MessageTemplate" → "Use RealInverseSeries[expression,{x,y},exactCutoff,options] with unassigned symbols x
  ,y."|>];

InverseResidual[result_Association, requested_: Automatic] := Catch[
  Module[{needed, p, c, ell, deltas, polys, unit, y, A, cut, r, expr, data},
    needed = {"InternalData", "OutputVariable", "NormalizedCutoff", "Assumptions"};
    If[(And @@ (KeyExistsQ[result, #] & /@ needed)),
      fail["Result", "Supply an unmodified result from RealInverseSeries."]];
    Block[{$assumptions = result["Assumptions"]},
      data = result["InternalData"];
      If[!MatchQ[data, {_, _, _Symbol, _List, _List}],
        fail["Result", "The result's InternalData is invalid."]];
      {p, c, ell, deltas, polys, unit} = data;
      y = result["OutputVariable"];
      cut = If[requested === Automatic, ce[1 + result["NormalizedCutoff"]]/p, requested];
      If[!exactRealQ[cut] || cmp[cut, 1] <= 0,
        fail["ResidualCutoff", "The residual cutoff must be an exact real constant greater than 1."]];
      A = ce[p (cut - 1)];
      r = spAdd[spCompose[modelTerms[p, deltas, polys],
        spAdd[unit, {{0, -1}}, A], A, ell], {{0, -1}}, A];
      expr = Total[(c (y/c)^ce[(p + #[[1]])/p]
        (#[[2]] /. ell → Log[y/c]/p)) & /@ r];
      <|"Expansion" → expr, "ZeroBelowCutoff" → (r === {}),
        "Cutoff" → cut, "RelativeResidualBlocks" → r,
        "Meaning" → "Formal expansion of the supplied finite model's forward residual; only powers below
        Cutoff are displayed. Input remainder bounds are not evaluated."|>
    ]
  ], $failureTag];

```

```

InverseResidual[____] := Failure["Arguments", <|
  "MessageTemplate" → "Use InverseResidual[result] or InverseResidual[result,exactResidualCutoff].">];

PerturbativeInverse[phi_, h_, {x_Symbol, y_Symbol, eps_Symbol}, n_Integer?((# >= 0) &)] :=
Module[{hp, dp, result = phi, k},
  If[Length[DeleteDuplicates[{x, y, eps}]] != 3,
    Return[Failure["Variables", <|"MessageTemplate" → "Use three distinct unassigned symbols.">]]];
  If[!FreeQ[phi, x] || !FreeQ[phi, eps] || !FreeQ[h, y] || !FreeQ[h, eps],
    Return[Failure["Variables", <|"MessageTemplate" → "phi must be independent of x,eps and h must be
      independent of y,eps.">]]];
  hp = h /. x → phi; dp = D[phi, y];
  Do[result += (-eps)^k D[hp^k dp, {y, k - 1}]/Factorial[k], {k, 1, n}];
  Expand[result]
];

PerturbativeInverse[____] := Failure["Arguments", <|
  "MessageTemplate" → "Use PerturbativeInverse[phi,h,{x,y,eps},nonnegativeInteger].">];

End[];
EndPackage[];

```

References

- [1] Vladimir Reshetnikov, *How to get an asymptotic of the real-valued branch of the inverse function?* Mathematica Stack Exchange, question 236367, December 2020. [Public question and discussion](#). The uploaded Markdown/PDF archive supplies the same two examples.
- [2] NIST Digital Library of Mathematical Functions, §1.10(vii), *Lagrange inversion theorem*. <https://dlmf.nist.gov/1.10#vii>. Consulted September 7, 2026.
- [3] Ira M. Gessel, *Lagrange Inversion*, 2016. <https://arxiv.org/abs/1609.05988>. In particular, Theorem 2.1.1 gives the classical coefficient identity.
- [4] Erlang Surya and Lutz Warnke, *Lagrange Inversion Formula by Induction*, 2023. <https://arxiv.org/abs/2305.17576>. An elementary formal-series treatment complementary to the local analytic argument used here.
- [5] Vladimir Reshetnikov, ProveIt repository, *Combinatorial Transseries and Their Inverses*, chapter “Coefficient calculus and the inversion engines.” [Companion source in the supplied series-and-transseries directory](#). Relevant phase-coordinate, convergence, collision, and residual sections consulted September 7, 2026.
- [6] Vladimir Reshetnikov, ProveIt repository, *Series and transseries*, group README. [Group overview, provenance, and scoped formalization inventory](#). Consulted September 7, 2026.
- [7] NIST Digital Library of Mathematical Functions, §5.11, *Asymptotic expansions of the gamma function*. <https://dlmf.nist.gov/5.11>. The leading Stirling approximation is used in the radius calculation.
- [8] G. A. Edgar, *Transseries for beginners*, Real Analysis Exchange **35** (2009/2010), 253–310. <https://arxiv.org/abs/0801.4877>.
- [9] Wolfram Research, *SeriesData*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/SeriesData.html>. Consulted September 7, 2026.
- [10] Wolfram Research, *InverseSeries*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/InverseSeries.html>. Consulted September 7, 2026.
- [11] Wolfram Research, *TestReportObject*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/TestReportObject.html>. Consulted September 7, 2026.